CS 61A — Week 3

Solutions to the handout problems (code for the exercises is in `solutions/`)

P1.

1. `g(f)` is the function `lambda x: f(x, x + 1)`. `g(f)(10)` is `f(10, 11)`, which is `-1`. Displays `-1`.
2. `(lambda: 3)()` is 3; `(lambda x: x * 2)(4)` is 8. Displays `11`.
3. `k(5)` displays `5` and returns the value of `print(5)`, which is `None`. `None or 'done'` is `'done'`. The interpreter displays `'done'` (with quotes, since it is a string value).

P2. (a) Each lambda becomes a `def` that returns the next function:

```
def make(a):
    def f(b):
        def g(c):
            return a * b + c
        return g
    return f
```

(b) The `if` statement becomes a conditional expression: `pick = lambda test: lambda x, y: x if test(x) else y`.

P3.

1. `g = outer(5)`: frame `f1` binds the parameter `n` to `5`; the lambda has parent `f1`.
2. `caller(g)`: `caller`'s frame has `n = 100`, but `g`'s body looks in its own frame, then `f1`: `n` is `5`. Value `15`.
3. The second lambda is written in the global frame, so its parent is global. When it runs, `n` is the global `n`, which is `50` by then. Value `60`.

Displayed: `15 60`. `caller`'s `n = 100` is never used by either function.

P4.

1. `curry(sub)(10)(3)` is `sub(10, 3) = 7`.
2. `reverse(sub)(x, y)` is `sub(y, x)`. So `curry(reverse(sub))(10)(3)` is `sub(3, 10) = -7`.
3. `curry(reverse(sub))(1)` is the function $k \mapsto$ `sub(k, 1)` $= k - 1$. `summation(3, ...)` is $0 + 1 + 2 = 3$.

P5.

```
def count_where(n, cond):
    count = 0
    while n > 0:
        if cond(n % 10):
            count = count + 1
        n = n // 10
    return count

def has_digit(n, d):
    return count_where(n, lambda x: x == d) > 0
```

The lambda turns "is this digit equal to `d`?" into the one-argument shape `cond` needs; it finds `d` in its parent frame, the frame of `has_digit`.

E1 (published answers: (a) `1`; (b) `None = 0`; (c) `so`; (d) `None + None = zero?`).

1. `x, y = print(1 or 2), (3 and 0)`. Both right sides are evaluated first. `1 or 2` is `1`, so `print(1)` displays `1`: line (a). Its value `None` goes to `x`. `3 and 0` is `0`, which goes to `y`.
2. The last line is a call to `print` with four operands, evaluated left to right.
3. First operand `print(x, '=', y)`: displays `None = 0`, line (b); value `None`.
4. Third operand `f(print)`: `f` is `lambda g: g(z)`, so this is `print(z)`. `z` is looked up when the body runs: it is `'so'` now, not `'but'`. Displays `so`, line (c); value `None`.
5. The outer call is `print(None, '+', None, '= zero?')`: displays `None + None = zero?`, line (d).

E2 (published answers: (a) `k < n`; (b) `max(largest, abs(term(k + 1) - term(k)))`; (c) `lambda k: term(2 * k - 1)`).

1. What do the names mean? `k` is the index of the left term of the current pair; `largest` is the largest difference seen so far.
2. The pairs are $(1,2),(2,3),\ldots,(n-1,n)$, so `k` runs from `1` to `n - 1`: the condition is `k < n`.
3. No `if` is allowed, so the running maximum is updated with `max`: `largest = max(largest, abs(term(k + 1) - term(k)))`.
4. `odd_gap`: the sequence `term(1)`, `term(3)`, `term(5)`, … is the sequence $k \mapsto$ `term(2k - 1)` for $k = 1, 2, 3, \ldots$ There are `(n + 1) // 2` odd numbers up to `n`, which is the first argument already given. So blank (c) is the function `lambda k: term(2 * k - 1)`.

Check: `odd_gap(5, square)` is `gap` of the terms 1, 9, 25, whose largest difference is 16.